

Стандарт разработки на Python

Стандарт разработки на Python в Wasaby Framework основан на [PEP-8](#) за исключением соглашения имён платформенных методов. Для более подробного понимания рекомендуется ознакомиться с [исходным документом](#).

Исключения из PEP-8

Длина строки

Длина строки не должна превышать 120 символов.

Правила именования платформенных методов

Допустимо использование [CamelCase](#) в именовании методов [объектов бизнес-логики](#) и методов платформы, которые используются в Python-модулях.

Например:

```
1 Account.ListOfCertificatesAndApplications(myfilter)
2 sbis.BLObject("Account").Invoke("ListOfCertificatesAndApplications",
  myfilter)
```

Правила оформления

Отступы

- Используйте четыре пробела для каждого уровня структурного отступа.
- Отступ элементов, обернутых в скобки, должен быть на том же уровне, что и элементы предыдущего уровня.

```
1 # хорошо
2 foo = long_function_name(var_one, var_two,
3                             var_three, var_four)
4 # хорошо
5 def long_function_name( # Для параметров функций нужно использовать
6     var_one, var_two, var_three,
7     var_four):
8     print(var_one)
9 # хорошо
10 foo = long_function_name(
11     var_one, var_two,
12     var_three, var_four)
1
2 # плохо
3 foo = long_function_name(var_one, var_two,
4                             var_three, var_four)
5 # плохо
6 def long_function_name(
7     var_one, var_two, var_three,
8     var_four):
9     print(var_one)
```

Пустые линии

- Вокруг блоков кода верхнего уровня (определения функций и классов) должно быть две строки разделения.
- Методы внутри класса разделяются одной строкой.
- Строку-разделитель внутри функций и методов можно использовать строго для выделения логических секций кода.

```
1 class Foo:
2     def __init__(self):
3         pass
4
5     def my_method(self):
6         pass
7
8
9 class Bar:
10     pass
```

Imports

- Конструкции импорта должны находиться в начале файла: после комментариев, docstrings и перед объявлением глобальных переменных и констант.
- Импорт каждого пакета из модуля должен быть на отдельной строке.

```
1 # хорошо
2 import os
3 import sys
4
1 # плохо
2 import sys, os
```

- Абсолютные пути импорта предпочтительнее (см. [PEP8](#)).

```
1 import mypkg.sibling
2 from mypkg import sibling
3 from mypkg.sibling import example
```

- Использование подстановочного импорта недопустимо.

```
1 # хорошо
2 import sbis
3 from sbis import RecordSet
4 record = sbis.Record()
5
1 # плохо
2 from sbis import *
```

Кавычки

- Какие кавычки использовать для выделения строк зависит от программиста.
- Для выделения многострочных строк всегда используйте три двойные кавычки.

```
1 # хорошо
2 """Длинная
3     Строка"""
4
1 # плохо
2 '''Длинная
3     Строка'''
```

Пробелы в выражениях и конструкциях

Последовательности

Пробелы в последовательностях нужно ставить только после разделительной запятой или двоеточия в словарях. Также можно ставить пробелы вокруг оператора выделения последовательности.

```
1 # хорошо
2 dct['key'] = lst[index]
3 spam(ham[1], {eggs: 2})
4 ham[1:9], ham[1:9:3], ham[:9:3], ham[1::3], ham[1:9:]
5 ham[lower+offset : upper+offset]
6 ham[: upper_fn(x) : step_fn(x)], ham[:: step_fn(x)]

1 # плохо
2 dct ['key'] = lst [index]
3 spam( ham[ 1 ], { eggs: 2 } )
4 ham[1: 9], ham[1 :9], ham[1:9 :3]
5 ham[ : upper]
```

Конструкции

- После условного выражения перед двоеточием не должно быть пробела.
- Между именем функции и скобками с аргументами не должно быть пробела.
- Не должно быть лишних пробелов в операциях присваивания.

```
1 # хорошо
2 if x == 4: print x, y; x, y = y, x
3 spam(1)
4 x = 1
5 y = 2
6 long_variable = 3

1 # плохо
2 if x == 4 : print x , y ; x , y = y , x
3 spam (1)
4 x           = 1
5 y           = 2
6 long_variable = 3
```

Математические операции и операции сравнения

Всегда должны быть окружены одиночными пробелами: операторы присваивания (=, +=, -= и т.д.), операторы сравнения (==, <, >, !=, <>, <=, >=, in, not in, is, is not, and, or, not).

Если используются операторы с различным приоритетом, то операции с более низким приоритетом должны быть окружены пробельным символом. Количество пробелов для выделения операции с каждой стороны не может превышать одного.

```
1 # хорошо
2 i = i + 1
3 submitted += 1
4 x = x*2 - 1
5 hypot2 = x*x + y*y
6 c = (a+b) * (a-b)

1 # плохо
2 i=i+1
3 submitted +=1
```

```
4 | x = x * 2 - 1
5 | hypot2 = x * x + y * y
6 | c = (a + b) * (a - b)
```

Функции

- Не используется пробел в описании значения по умолчанию для аргументов функции.
- Используется пробел в описании значения по умолчанию с указанием типа аргумента и вокруг конструкции "->".

```
1 | # хорошо
2 | def complex(real, imag=0.0):
3 |     return magic(r=real, i=imag)
4 | def munge(sep: AnyStr = None):
5 | def munge() -> AnyStr:
1 | # плохо
2 | def complex(real, imag = 0.0):
3 |     return magic(r = real, i = imag)
4 | def munge(input: AnyStr=None):
5 | def munge(input: AnyStr)->PosInt:
```

Комментарии

1. В начале каждого файла следует указать его назначение.
2. В файле с исходным кодом должны быть прокомментированы все классы и методы.
3. Для вынужденных решений, когда код осознанно написан "плохо", следует добавить ссылку на Задачу/Ошибку, после решения которой код можно улучшить. В результате разработчик может оперативно понять причину и отследить решение вопроса.
4. Для документирования API используйте синтаксис [docstrings](#), а также рекомендации из [статьи](#).

Однотрочный комментарий

1. Начинается с символа #, перед символом минимум два пробела, а после — один.
2. Имеет такой же структурный отступ, как и сам код.

```
1 | # плохо
2 | # компенсация для границы
3 | x = x + 1
4 |
5 |     # компенсация для границы
6 | x = x + 1
1 | # хорошо
2 | x = x + 1 # компенсация для границы
```

Правила именования

Пакеты и модули

- Модули должны содержать в имени только строчные символы. Подчёркивания в именовании должны присутствовать для лучшей читаемости.
- Пакеты в имени так же должны состоять из строчных букв, но знак подчёркивание нежелателен.
- Модуль Python, написанный на C/C++, может начинаться со знака подчёркивания (пример: `_socket`).

Классы

Имена классов должны писаться в [CamelCase](#) стиле. Например: MyClassName.

Исключения

Соглашение именования исключений такое же как и для классов. Тем не менее имя исключения должно заканчиваться словом "Error".

Функции

Функции и методы должны быть написаны строчными символами. Если название состоит из двух или более слов, то они должны быть разделены подчёркиванием.

Например:

```
1 def my_method_name(self):  
2     pass
```



Исключение - модуль sbis.

Аргументы функций и методов

- Используйте всегда `self` для именования первой переменной метода класса.
- Используйте всегда `cls` для именования первой переменной классического метода.
- Если название аргумента соответствует зарезервированному слову, то лучше добавить знак подчёркивания в конец.

Методы и переменные экземпляров

- Имена методов и экземпляров должны быть написаны строчными символами. Если название состоит из двух или более слов, то они должны быть разделены подчёркиванием.
- Используйте знак подчёркивания в качестве префикса, для указания непубличных методов и переменных.
- Используйте два знака подчёркивания в качестве префикса, для приватных методов.

Константы

Константы обычно описываются в начале модуля и должны состоять из прописных символов. Если наименование состоит из нескольких слов, то они должны быть разделены знаком подчёркивания.

Например: MAX_OVERFLOW

Статический анализатор кода

Вывод всех файлов с повышенной цикломатической сложностью

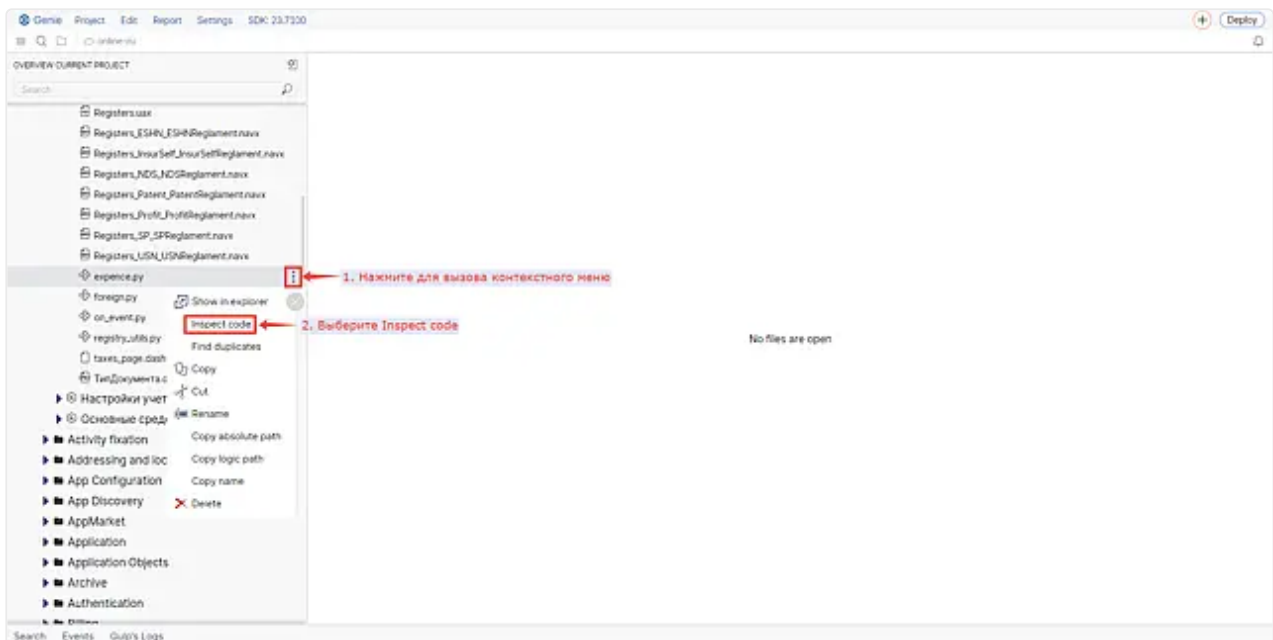
```
1 set RADONFILESENCODING=utf-8-sig  
2 radon cc -nc -as <путь>
```

Запуск статического анализа python-файлов

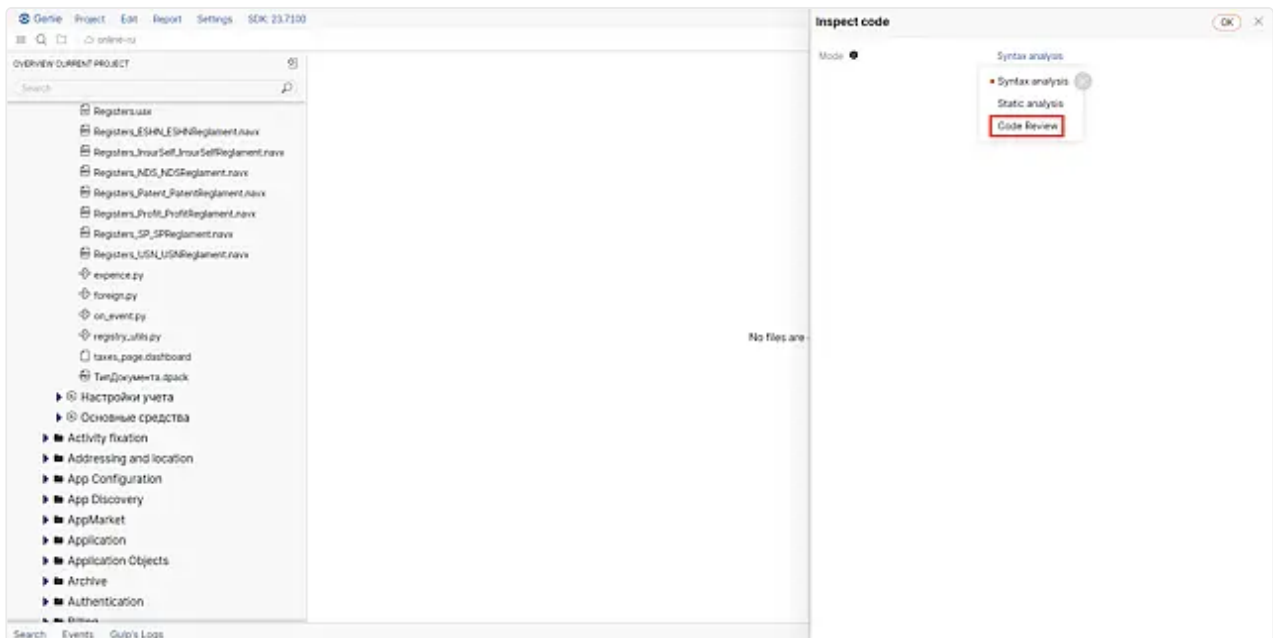
Запуск статического анализатора из Genie

Чтобы проверить файл с помощью [статического анализатора из Genie](#):

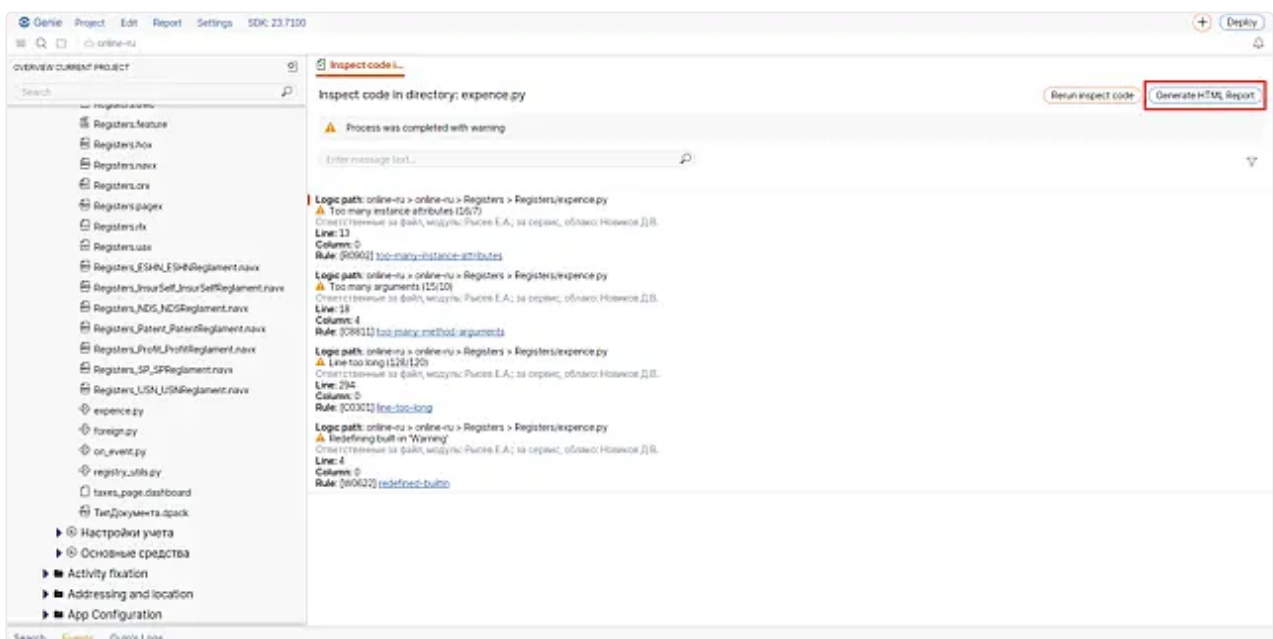
1. В контекстном меню файла выберите пункт "Inspect code".



2. В окне "Inspect code" выберите режим "Code Review" и нажмите "OK".



3. Для загрузки отчета о проверке в файл нажмите кнопку "Generate HTML Report":



Если надо проверить файл отдельно от проекта, например, когда выгрузить образ проекта целиком не получается, то проверка выполняется только с [командной строки](#).

Необходимо иметь в виду, что если не указан /project=, то СТАН не может анализировать все зависимости того, что проверяет, и в этой ситуации часть правил, связанных с зависимостями отключается, т.е. проверка в отрыве от контекста будет менее полной.

Критерии оценки качества кода

[См. документ](#)

Настройка проверки корпоративного стандарта кода в PyCharm

[См. статью](#)